

Long-Context Attention: Distributed Dense, Windowed, Block-Sparse Kernels

CME 213 Final Project Report

Emma Sampietro (esampi@stanford.edu) – Arturo Favara (favara@mit.edu)

1 Problem: Long-Context Attention

Scaled dot-product attention [1] maps queries, keys, and values $Q, K, V \in \mathbb{R}^{H \times N \times d}$ to an output $O = \text{softmax}(QK^\top / \sqrt{d})V \in \mathbb{R}^{H \times N \times d}$, where H is the number of heads, N is the sequence length, and d is the head dimension. If the $N \times N$ score matrix $QK^\top$ (the pairwise inner products of every query with every key) is materialized, the cost is $O(HN^2d)$ FLOPs, occupying $O(N^2)$ memory. For N in the tens of thousands, the quadratic term dominates both wall time and on-chip memory, which is why long-context attention is a current bottleneck for LLM training and inference [2]. Multi-GPU parallelism is therefore required: at $N=32\text{K}$, $H=16$, $d=64$ (GPT-2 Medium scale [3], large enough for asymptotic effects to dominate), our single-GPU dense kernel takes 2.85s and the four working tensors total $4NHd = 514\text{MB}$ on a 24GB device. This could be tractable for one rank, but quickly becomes prohibitive once a batch dimension is added.

The standard response is to combine structured sparsity with multi-GPU parallelism [4]. We study three sparsity patterns on a single GPU and across four GPUs over MPI: a dense exact kernel in the FlashAttention style [5], a sliding-window kernel with half-width w [6], and a block-sparse kernel driven by a Compressed Sparse Row mask [7]. All inputs and outputs are FP32 at $d=64$ unless stated otherwise. Our central question is:

At what N does structured sparsity beat an IO-aware dense kernel in wall time, and how does each compare to PyTorch’s production attention SDPA (scaled_dot_product_attention)?

Since speed is only useful if the output is still meaningful, we additionally evaluate the approximation quality of the sparse outputs (Fig. 4, with details in Appendix E).

2 Background and Related Work

Exact IO-aware attention. FlashAttention [5] avoids materializing the $N \times N$ score matrix by tiling and using an online softmax [8], which reduces high-bandwidth memory (HBM) traffic to $O(N^2d/\sqrt{M})$. FlashAttention-2 [9] improves warp-level partitioning and is the backend PyTorch’s SDPA [10] dispatches to on Ampere+ hardware. On the Turing GPU we used [11], FA-2 is not supported, and SDPA in FP16 dispatches to PyTorch’s memory-efficient backend instead, which also uses tiling and an online softmax but is engineered differently. All of these techniques are single-GPU. For multi-GPU execution, Ring Attention [12] extends exact attention across devices, but it is designed for dense workloads only.

Structured sparse attention. Longformer [6] replaces the dense pattern with a window of half-width w plus G globally-attended tokens, which reduces per-query work to $O((2w+G)d)$. BigBird [7] adds random long-range links on top. Both papers report FLOP reductions but rarely measure GPU wall-clock speedups on real hardware, and that is the gap we set out to close.

Our variants. The three kernels we implement share an identical FA-style fused-tile, register-resident online-softmax skeleton, and differ only in their K/V iterator. This maintains a fair comparison: differences in measured time come from the sparsity pattern itself, not differences in kernel engineering. We compare against PyTorch SDPA-FP32 (the math backend, our precision) and SDPA-FP16 (the strongest production reference available on our hardware).

3 Implementation: Kernels and MPI

Single-GPU kernel. Each kernel uses one CUDA block per $B_r=64$ -row query tile, and one thread per query row. The per-row online-softmax state (a running max m_i , a running normalizer ℓ_i , and a partial output $O_i[d]$) stays in

registers, so there is no inter-thread reduction. To avoid SMEM bank conflicts, we pad the row stride from $d=64$ to $d_p = d+1 = 65$: with 32 banks of 4 bytes each, the off-by-one stride spreads consecutive rows across different banks, so 32 threads reading the same column hit 32 distinct banks instead of all colliding on one. The full dynamic shared-memory (SMEM) tile holds $(B_r + 2B_c)d_p$ floats [11]. We opt into the >48 KB SMEM allowance via the dynamic-SMEM attribute and reach 49 KB on the dense kernel. The inner loop is $s_j = Q_i \cdot K_j / \sqrt{d}$, followed by an online-softmax update of (m_i, ℓ_i) and an accumulation $O_i += e^{s_j - m_{\text{new}}} V_j$. To bring K/V tiles into shared memory efficiently, each warp issues one wide 512-byte `float4` load per tile (4 floats per thread). The inner arithmetic uses CUDA’s `rsqrtf` and fast-exp hardware intrinsics, which we accept as slightly less precise in exchange for higher throughput. The 3 kernels differ only in their K/V iterator: dense streams every tile, windowed restricts to tiles intersecting $[i-w, i+w]$ plus an optional global-token range, and block-sparse iterates over the active block-columns enumerated by a CSR `row_ptr/col_idx` (Algorithm 1 and iterators in Appendix I).

Head-parallel *head-par* MPI. Since heads are independent, the simplest decomposition distributes them across ranks. The driver does three Scatter calls for Q, K, V and one Gather for O , and everything else is per-rank. Per-iteration communication is $4NHd/P$ bytes per rank (kernel-agnostic), and CUDA-aware MPI sends device pointers directly with no host staging. This decomposition is correct for any kernel, but it throws away all of the locality structure that windowed and block-sparse provide.

Sequence-parallel *seq-par* windowed MPI. For the windowed kernel, head-parallel communication is wasteful: compute is only $O(NwHd)$, so when $w \ll N$, communication dominates. We exploit windowed locality instead: rank r owns rows $[rN/P, (r+1)N/P)$ of Q and O , plus halos of width w in K/V from each neighbor. A query at row i only touches K/V in $[i-w, i+w]$, so per-iteration communication is $4 \cdot 2wHd$ bytes (four `Sendrecv` calls, one per neighbor-direction

for each of K and V), which is *independent of N* . Because heads are interleaved in the $[H, N_k^{\text{loc}}, d]$ buffer, the halo edges are packed into contiguous device buffers via `cudaMemcpy2D` before each exchange. The *seq-par* kernel takes a local-offset parameter equal to the left halo width to translate global query rows into the local K frame. We chose halo exchange over a full K/V All-gather (which moves the same volume as head-parallel) and Ring-Attention’s blockwise rotation (as dense-only) [12]. Algorithm 2 in Appendix I shows the per-iteration structure.

4 Performance Model

Roofline. For each kernel, achieved performance is bounded by $\min(\pi, \beta I)$ [13], where π is FP32 peak throughput (16.3 TFLOPS on the RTX 6000), β is DRAM bandwidth (624 GB/s), and I is arithmetic intensity (FLOP/byte). With d -element FP32 stores at byte size $B=4$,

$$I_{\text{dense}} \approx N/B, \quad I_{\text{win}} \approx 2w/B, \quad I_{\text{bsp}} \approx \rho N/B.$$

The ridge sits at $I^* \approx 26$ FLOP/byte. At $N=4K$ we predict $I \approx 1024$ for dense, 128 for windowed at $w=256$, and 102 for block-sparse at $\rho=0.1$, all of which sit deep in the compute-bound regime.

Communication cost. We use the standard $\alpha + \beta n$ model [14], where α is the per-message latency, β is the per-byte transfer cost, and n is the message size. Head-parallel moves $n_{\text{hp}} = 4NHd/P$ bytes per iteration, while *seq-par* moves $n_{\text{sp}} = 8wHd$. The ratio $n_{\text{hp}}/n_{\text{sp}} = N/(2Pw)$ predicts a $4\times$ communication reduction at $N=4K$, $P=4$, $w=256$, and the advantage grows linearly with N .

Amdahl and isoefficiency. If a fraction s of work is the (sequential) communication tail, the P -rank speedup is $1/(s + (1-s)/P)$ [15]. For *seq-par* windowed, n_{sp} is small, so s is small and the kernel should scale well. For head-parallel windowed, n_{hp} dominates the cheap compute, so s is large and scaling should fail. The isoefficiency function for *seq-par* follows from communication $\sim wHd$ versus compute $\sim NwHd/P$: efficiency E stays constant when $N \sim P$, so $W \sim N \sim P$. For head-parallel windowed, the communication itself grows with N , so there is no N at which the kernel beats communication at fixed w .

5 Benchmarking Methodology

We time each kernel with CUDA events, reporting the median of 10 measured iterations after 5 warmups. The multi-head dispatcher runs each head on its own stream (up to 16), and stream-sync bounds the timed region. For MPI we additionally bracket the loop with a Barrier and use MPI’s wall-clock timer. Peak device memory is sampled with `cudaMemGetInfo` both before any allocation and after the first measured iteration, so that lazy kernel-side allocations are captured. Nsight Systems gives per-rank timelines, and Nsight Compute gives FLOP, bytes, and time for the Roofline. Inputs are filled deterministically and measurement variance is consistently small (Appendix A). All runs are on Stanford ICME’s `gpu-turing` partition with 4× Quadro RTX 6000 per node (Turing, $\pi=16.3$ TFLOPS FP32, $\beta=624$ GB/s HBM), at $H=16$, $d=64$, FP32 unless otherwise stated. As a reference we compare against PyTorch 2.5.1 SDPA in both FP32 (the math backend, our precision) and FP16 (PyTorch’s memory-efficient tensor-core backend on Turing; FA-2 itself requires Ampere or newer).

6 Diagnosing the Bottleneck

Figure 1 plots achieved performance at $N=4096$, $H=16$. Our three FP32 kernels (dense, windowed, and block-sparse, respectively) reach 13.4%, 7.5%, and 13.4% of FP32 peak. The model’s compute-bound prediction is confirmed (all points sit deep right of the ridge), but the fraction-of-peak number tells us where the real ceiling is: it is the FP32 CUDA-core throughput on Turing, not bandwidth. Two factors visible in Nsight Compute reinforce this. First, a per-thread score register buffer drives roughly 254 registers per thread, which caps theoretical occupancy at 4 blocks per streaming multiprocessor (SM, $\sim 25\%$ on Turing). This is as each SM has a fixed register file (65,536 32-bit registers on Turing), so high per-thread usage limits how many threads can be co-resident. Second, Turing’s tensor cores accept FP16 inputs only, so the FP32 path cannot use them. The PyTorch SDPA comparison makes the ceiling concrete. SDPA-FP32 (math backend) at

$N=4096$ runs in 10.6 ms versus our 31.4 ms post-variant dense (a $3.0\times$ gap), because SDPA reaches around 40% of FP32 peak through better register tile partitioning. SDPA-FP16 at 2.23 ms is another $5\times$ on top, almost entirely from tensor cores and reduced precision. The right path past the FP32 compute ceiling is therefore tensor cores or FA-2-style warp specialization, not further memory-side work.

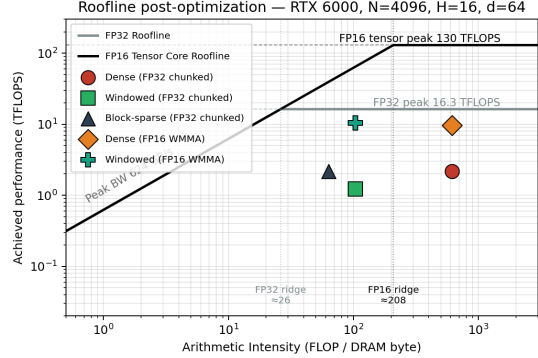


Figure 1: Achieved Roofline on RTX 6000 at $N=4096$, $H=16$. FP32 kernels sit against the 16.3 TFLOPS FP32 ceiling; FP16 WMMA points against the 130 TFLOPS tensor-core ceiling. All sit deep right of their respective ridges (compute-bound, not bandwidth-bound). The WMMA dense kernel reaches 7.5% of FP16 peak. TFLOPS are derived from wall-clock multi-head throughput (with stream overlap), not Nsight.

MPI bottlenecks. On rank 0 at $N=4096$, $P=4$, Nsight Systems reports per-iteration kernel time of 16.4 ms for head-par dense, 8.82 ms for head-par windowed, and 6.18 ms for seq-par windowed. The seq-par kernels are smaller (each rank processes N/P rows) and there are $P\times$ more launches, but the 16 streams overlap them effectively. An upper bound on MPI time is (wall – largest single kernel): 5.06 ms for head-par windowed versus 1.53 ms for seq-par, a measured $3.3\times$ communication reduction. The model predicted $24\times$ at this configuration; the gap is the latency tail (the α in $\alpha + \beta n$), which dominates the tiny seq-par messages.

7 Algorithmic Variants

We explored three classes of variants, each one motivated by a specific diagnostic from §6 and

each one shifting the bottleneck. The three kernel families exhibit the predicted asymptotic complexities, with measured log-log slopes of 1.95 (dense, $O(N^2)$), 1.05 (windowed, $O(N)$), and 1.89 (block-sparse at $\rho=0.1$, $O(\rho N^2)$); windowed overtakes block-sparse around $N \approx 8\text{K}$ (verified visually in Fig. 3, §8). The optimizations below close the constant-factor gap to PyTorch’s production references on top of these baseline curves.

Vectorized loads, chunked online softmax.

Nsight Compute flagged two obvious targets: 88% wasted memory sectors (32 threads pulling from 32 cache lines per warp instruction), and a 254-register-per-thread footprint driven by a score buffer of size B_c that lives across the whole tile. We applied two compounding fixes. First, `float4` cooperative tile loads, so each warp issues a 512B contiguous request that lands on 4 aligned cache lines. Second, a chunked online softmax that processes each tile in 4 chunks of $C_n=16$ columns, with one online-softmax correction per chunk. The combined effect at $N=4096$ is $1.45\times$ on dense (45.6 to 31.4ms), $1.38\times$ on windowed, $1.50\times$ on block-sparse. The chunked gain did *not* come from higher occupancy: Nsight reports that registers per thread fell only from about 254 to 243–252, and the occupancy cap stayed at 4 blocks per SM. It came from improved instruction-level parallelism: a smaller live register set per chunk lets the compiler interleave the dot products, corrections, and probability accumulations more aggressively.

Tensor cores via WMMA. §6 pinpointed FP32 compute throughput as the dominant ceiling, and pointed at tensor cores as the only way past it. We added FP16 variants of all 3 kernels using the Warp Matrix Multiply-Accumulate (WMMA) intrinsics from `nvcuda::wmma`, with m16n16k16 fragments, FP16 inputs, FP32 accumulators, four warps per block, and each warp owning 16 query rows. The SMEM budget was tight against Turing’s 64KB limit, so we unioned K and the post-softmax probability matrix P in shared memory (they are temporally disjoint), which dropped the working set from 74 KB to 63 KB. Speedups versus the FP32 chunked

baseline are essentially constant in N : dense $4.47\times$, windowed $6.65\text{--}9.61\times$ (across $N=1\text{K}$ to 32K), and block-sparse $4.30\times$. Achieved 9.7–10.5 TFLOPS is 7.5–8.1% of the 130 TFLOPS FP16 peak, and the gap to SDPA-FP16’s $\sim 24\%$ is the warp-specialized softmax used by both FA-2 and PyTorch’s memory-efficient backend, which is out of scope here. *This variant moves the bottleneck:* dense + WMMA is now FP16 tensor-core-bound rather than FP32 compute-bound, and windowed + WMMA, by removing both the compute bottleneck and most of the windowed kernel’s work, becomes decisively faster than SDPA-FP16 at long context (Fig. 3).

Sequence-parallel MPI driver. The §4 model predicted that seq-par would beat head-par on windowed at long N , because n_{sp} is independent of N . Measured: switching from head-par to seq-par at $P=4$ improves wall time from 7.26 ms to 1.92 ms at $N=4096$ ($3.8\times$), and the gap widens as N grows, exactly as the model predicted. We composed this win with the WMMA variant by propagating the windowed-FP16 kernel into the seq-par driver, and the WMMA implementation lift survives MPI ($3.14\times$ versus FP32 chunked at $N=16\text{K}$, $P=4$; full WMMA + MPI table in App H). Full scaling results in §8.

8 Scaling to Four GPUs

Strong scaling. Figure 2 shows the full strong-scaling picture across the kernel $\times$ precision $\times$ driver $\times$ head-dim matrix. Three patterns confirm the §4 predictions. (i) Seq-par windowed (solid lines) tracks near-ideal at every (N, d) , reaching **96% efficiency** at $d=128, N=16\text{K}$, $P=4$ for FP32 and 87% for FP16, which is our strongest scaling result. (ii) Head-par (dashed lines) flattens as soon as the kernel gets cheap relative to the $O(NHd/P)$ Scatter+Gather: at $N=4096$, head-par dense FP32 still climbs to $2.48\times$ at 4 ranks (62%), but head-par windowed FP32 only reaches $1.02\times$ (26%), because its compute is roughly $w/N \approx 1/16$ as large. (iii) The model predicted that FP16 block-sparse at small N should fail to scale at all, because communication is fixed but its kernel is the cheapest. Measured at $N=4096, d=64$: 2.30 ms at $P=1$ rises to 4.84 ms at $P=4$. It *anti-scales*. The opposite

Strong scaling across kernels, head dims, and drivers (H=16)

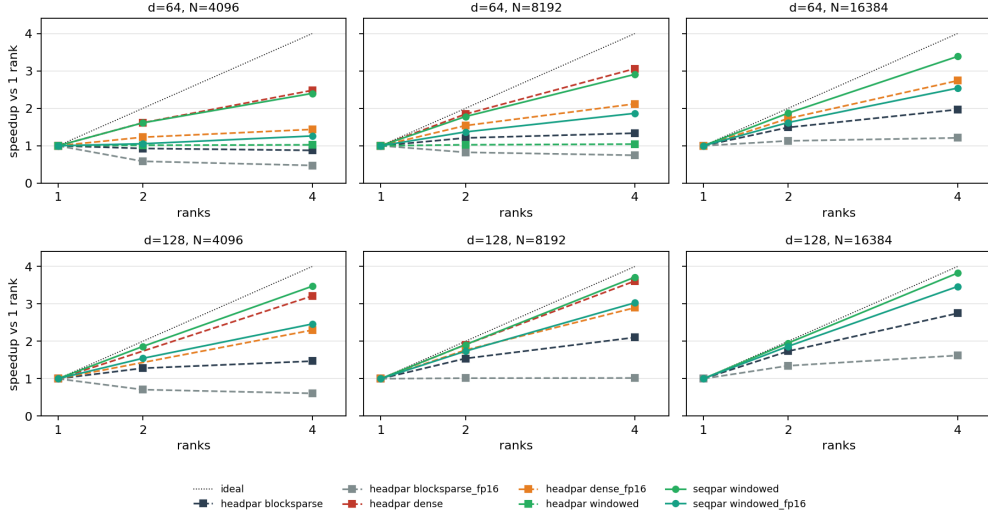


Figure 2: Strong scaling, $H=16, w=256, P \in \{1, 2, 4\}$, on full kernel $\times$ precision $\times$ driver $\times$ head-dim matrix. Solid = seq-par, dashed = head-par, dotted = ideal linear speedup. Rows: $d=64, d=128$.

extreme is equally clean: head-par dense FP32 at $d=128, N=8K$ hits 90% efficiency at $P=4$, because the dense kernel has so much compute that communication becomes a negligible fraction.

Weak scaling. Fixing per-rank work at 4 heads (dense head-par at $N=4096$), wall time grows from 9.58ms at 1 rank to 13.74ms at 4 ranks, a 43% slowdown that reflects the linear growth of scatter+gather volume in P , exactly as the head-parallel cost model predicts.

Isoefficiency confirmation. For seq-par WMMA windowed, measured efficiency at $P=4$ rises 32% to 47% to 64% as N grows from 4K to 8K to 16K, exactly the $W \sim N \sim P$ curve from §4. This rising-efficiency curve is exactly the predicted isoefficiency pattern: a faster kernel spends proportionally more time in communication at fixed N , and recovers efficiency only as N grows.

Comparison to a production reference. Figure 3 shows our kernels against SDPA on the same shapes. SDPA-FP16 stays roughly 3 $\times$ ahead of our WMMA dense, because of the warp-specialization techniques (memory-efficient backend on Turing) that we do not reproduce. Our *windowed*-WMMA, however, beats SDPA-FP16 *decisively*. The crossover sits at

$N \in [1024, 2048]$, and the lead grows monotonically: 2.7 $\times$ at $N=4K$, 5.3 $\times$ at $N=8K$, 10.4 $\times$ at $N=16K$, 21.3 $\times$ at $N=32K$, and a measured **49.5 $\times$** at $N=128K$ versus SDPA-FP16’s 2123 ms at the same size. Across 4 GPUs the lead compounds: at $N=16K$ our seq-par WMMA windowed runs in 1.56 ms versus SDPA-FP16’s 33.3 ms on a single GPU, a 21.3 $\times$ lead at $P=4$ on a problem where SDPA cannot use the additional GPUs at all. At long context, *algorithmic sparsity dominates implementation efficiency by a wide margin*, and combining both with multi-GPU parallelism gives a strict improvement over the strongest production reference available.

9 Correctness

Each single-GPU kernel is validated against a NumPy reference at small N , over a ladder of test cases that step through powers of 2 plus selected edge cases. Dense max absolute error is $4.2\text{--}6.0 \cdot 10^{-7}$ across 4 cases (including a causal variant); windowed is $3.3\text{--}9.2 \cdot 10^{-7}$ across 6 cases (including $w \geq N, w=0, G>0$, and causal); and block-sparse with $\rho=1.0$ reproduces dense at FP32 epsilon. All errors are four orders of magnitude inside our 10^{-3} tolerance, and the WMMA variants verify within the FP16 envelope (around 10^{-4}). **Multi-rank consistency.** Head-parallel

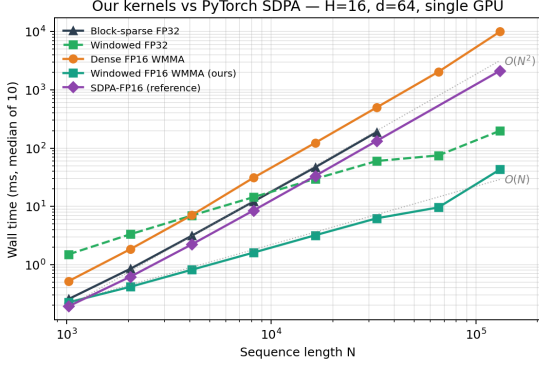


Figure 3: Our kernels versus PyTorch SDPA, single GPU. Block-sparse and dense follow $O(N^2)$; windowed follows $O(N)$. Our windowed FP16 WMMA crosses below SDPA-FP16 at $N \approx 2K$ and reaches $49.5\times$ at $N=128K$ (Appendix C).

at $P \in \{1, 2, 4\}$ is bit-identical to the single-GPU binary at the same seed ($\text{max_abs} = 0$). Seq-par at $P > 1$ yields $\text{max_abs} \approx 7.7 \cdot 10^{-7}$ versus single-GPU, which is at the level of FP32 rounding because the halo changes where tile boundaries fall, which changes the order in which the online softmax accumulates, and floating-point addition is not associative.

10 Discussion

Verdict on our hypotheses. We expected windowed to dominate at long N both on time *and* on memory. The time half is confirmed ($18\times$ at $N=16K$, $36\times$ at $N=32K$ versus our dense). The memory half is *refuted*: all three kernels measure within 1% of each other at every N , because the $4NHd$ footprint of Q, K, V, O dominates the BlockCSR indirection (Appendix D). We also expected block-sparse to win at low ρ and large N ; instead it wins at *small* N because of its low constant factor, and is overtaken by windowed asymptotically. Worse, on approximation quality (Fig. 4, details in Appendix E), the random block-sparse mask stays near $\ell_2 \approx 1$ across all three input regimes, while windowed attention reaches $\ell_2 = 0.025$ on locally-correlated data. The takeaway is that the *structure* of sparsity matters more than sparsity alone: windowed exploits a real locality signal, while a random mask exploits nothing. The block-sparse machinery only pays off with an

informed mask (a Longformer-style local-plus-global CSR), which our kernel supports as a drop-in but which we did not benchmark.

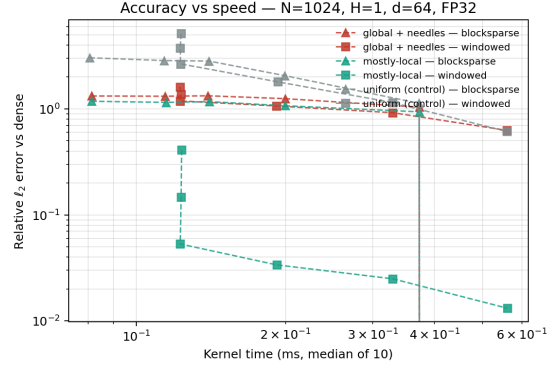


Figure 4: Approximation quality versus speed at $N=1024, H=1, d=64$. Squares = windowed, triangles = block-sparse; colors mark three input regimes (mostly-local, global+needles, uniform). Only windowed on locally-correlated data reaches $\ell_2 < 0.1$; random block-sparse stays near $\ell_2 \approx 1$ regardless of density.

What gave the largest payoff. Sequence-parallel MPI for the windowed kernel was the single largest improvement (roughly $3.8\times$ at 4 ranks). WMMA on the windowed kernel was the second ($6.6\text{--}9.6\times$). Both stack cleanly: the seq-par + WMMA composition reaches 96% efficiency at $d=128, N=16K$. The smallest payoff was the non-blocking halo overlap, which gave $1.19\times$ at $N=4K$ and parity at $N=16K$ (App F). The model predicts exactly this, since overlap only helps when βn is comparable to compute, and not when compute already dominates.

Limitations. We did not implement the backward pass, and we did not adopt the warp-specialized softmax behind SDPA-FP16’s residual $\sim 3\times$ dense-FP16 gap. The `flash-attn 2.x` package requires Ampere or newer, so we could not benchmark FA-2 standalone on Turing. Our work is single-node; across nodes, point-to-point latency would grow, but the halo-exchange locality argument still holds.

Future work. There are two exciting directions: drop our windowed kernel into LLM training to see whether the ℓ_2 error hurts model quality, and layer the windowed iterator on an FA-2 backbone for compounded speedups.

Acknowledgments

This project made use of Anthropic’s Claude Opus 4.7 as a coding and writing assistant. The use included kernel design discussions, CUDA debugging, plot script generation, SLURM scripting, and iterative report proofreading. All code, design decisions, and reported measurements were verified by the authors.

References

- [1] Ashish Vaswani et al. Attention is all you need. In *Advances in Neural IPS*, volume 30, 2017.
- [2] Reiner Pope et al. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, volume 5, 2023.
- [3] Alec Radford et al. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- [4] Yi Tay et al. Efficient transformers: A survey. *ACM Computing Surveys*, 55(6):1–28, 2022.
- [5] Tri Dao et al. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural IPS*, volume 35, 2022.
- [6] Iz Beltagy et al. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [7] Manzil Zaheer et al. Big bird: Transformers for longer sequences. In *Advances in Neural IPS*, volume 33, 2020.
- [8] Maxim Milakov et al. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.
- [9] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [10] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural IPS*, volume 32, 2019.
- [11] NVIDIA Corporation. NVIDIA Turing GPU architecture: graphics reinvented. Technical report, NVIDIA, 2018.
- [12] Hao Liu et al. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*, 2023.
- [13] Samuel Williams et al. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [14] Roger W. Hockney. The communication challenge for MPP: Intel Paragon and Meiko CS-2. *Parallel Computing*, 20(3):389–398, 1994.
- [15] Gene M. Amdahl. Validity of the single proces-

sor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.

Appendices

A Measurement methodology

Input generation. Inputs Q , K , V are filled deterministically from a fixed-seed linear congruential generator producing normal-range floating-point values (avoiding special cases like all-zeros or subnormals, which can trigger hardware fast paths), so every kernel sees the same data at every shape.

Variance. The min-max spread within the 10 measured iterations stays under 1% of the median for almost all sweeps and never exceeds 5% (at the smallest N , where event resolution becomes comparable to kernel time), so we report the median only and do not plot error bars.

B Generality to $d=128$

All six kernels (dense, windowed, and block-sparse, in both FP32 and FP16 WMMA) extend to the production-typical head dimension $d=128$. The only change is in the host wrapper: $B_r = B_c = 32$ instead of 64, so that the $(B_r + 2B_c)d_p \cdot 4$ bytes for the $Q/K/V$ tiles plus a dual-use FP32 scratch ($\max(B_c, d)$ wide) fits Turing’s 64 KB SMEM limit. The kernel templates themselves are unchanged. At $N=4096, H=16$: dense, windowed, and block-sparse FP32 measure 187.4, 25.2, and 19.1 ms, and the WMMA variants measure 49.1, 6.7, and 5.3 ms. The asymptotic ordering, log-log slopes, and cross-kernel ratios all match the $d=64$ figure within a constant. WMMA lifts at $d=128$ are $3.6\text{--}3.8\times$, slightly below $d=64$ ’s $4.3\text{--}6.6\times$ because the smaller B_r reduces per-launch tensor-core utilization.

C Extension to $N=131,072$ and L2 spillover

Pushing the sweep to $N=131,072$ at $H=16, d=64$: our windowed-WMMA measures 42.9 ms, SDPA-FP16 measures 2123 ms, and the ratio is $49.5\times$ (actual, not extrapolated). SDPA’s curve continues to match its $O(N^2)$ extrapolation to within 0.3% (2128 ms predicted from $N=32\text{K}$). Our windowed-WMMA grows $4.5\times$ for a $2\times$ problem from $N=64\text{K}$, abandoning the sub-linear regime that held at $N \leq 64\text{K}$: the per-launch working set (around 2 GB of K/V) starts exceeding L2 reuse on the RTX 6000. The asymptotic ordering ($O(N)$ versus $O(N^2)$) is intact, and the constant factor on our windowed kernel just rises modestly past $N=64\text{K}$.

D Peak memory comparison

N	Dense	Windowed	Block-sparse	Max Δ
4,096	66 MB	66 MB	68 MB	+3.0%
16,384	258 MB	258 MB	260 MB	+0.8%
32,768	514 MB	514 MB	516 MB	+0.4%
65,536	1026 MB	1026 MB	—	$\sim 0\%$

Table 1: Peak GPU memory ($H=16, d=64$, FP32) via `cudaMemGetInfo`. The $4NHd$ footprint of Q, K, V, O dominates, and the BlockCSR indirection costs at most a few MB even at small N .

E Approximation quality

We measured the relative ℓ_2 error of each sparse kernel against the FP32 dense reference at $N=1024, H=1$, across three synthetic input regimes: *mostly-local* (Q/K from a Cholesky-factored exponential-decay covariance, which models locally-correlated text), *global+needles* (a few strong global tokens plus planted random long-range pairs), and *uniform* (IID Gaussian, no exploitable structure). Windowed at $w=64$ on mostly-local data achieves $\ell_2 = 0.05$ in 0.12 ms, and at $w=256$ it pushes to $\ell_2 = 0.025$. On global+needles it saturates around 0.9 regardless of w , because no choice of local window can capture the planted long-range pairs. *Random block-sparse masks are essentially useless* on all three input regimes: $\ell_2 \approx 1$ on mostly-local, and $\ell_2 \approx 3$ on uniform. This is the gap that a designed Longformer-style local-plus-global mask would close, and our CSR kernel supports it as a drop-in.

F Non-blocking halo overlap

We added a non-blocking halo path: non-blocking sends and receives for K and V , then inner-region kernel launches (rows $[w, N_{\text{loc}}-w)$) that do not touch halo data, then a wait, an unpack step, and boundary kernels for the rows that do.

N	serial (ms)	overlap (ms)	speedup
4096	2.41	2.03	$1.19\times$
8192	4.16	3.81	$1.09\times$
16384	7.32	7.25	$1.01\times$

The overlap win shrinks as N grows because halo volume is fixed in N while inner compute grows linearly, exactly consistent with our $\alpha + \beta n$ model.

G End-to-end attention layer

Our $21\times$ kernel-only headline is, by construction, a kernel-only number. To check that it survives realistic wrapping, we exposed the dispatcher via an `extern "C"` shim so PyTorch can call it via `ctypes`, and benchmarked a full transformer-style layer (3 FP16 `nn.Linear` projections, then an FP32 cast at the kernel boundary, then our windowed-FP16 kernel, then an FP16 output projection) against the same projections wrapping PyTorch’s SDPA. The crossover sits at $N \approx 8K$ for both $d=64$ and $d=128$, and past it our layer pulls ahead: $5.2\times$ at $N=32K$, $d=64$, and $2.1\times$ at $N=16K$, $d=128$. The drop from $21\times$ (kernel-only) to $5.2\times$ (layer) at $N=32K$ is entirely projection overhead.

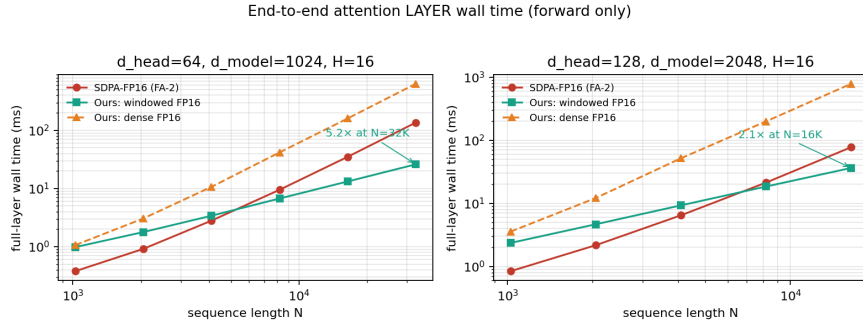


Figure 5: End-to-end attention layer wall time (forward only, $H=16$).

H Detailed WMMA + MPI scaling

config	1 rank	4 ranks	speedup	eff
Windowed FP32, $N=16K$	16.61	4.90	$3.39\times$	85%
Windowed FP16 WMMA, $N=16K$	3.97	1.56	$2.54\times$	64%
Windowed FP32, $N=4096$	4.09	1.74	$2.35\times$	59%
Windowed FP16 WMMA, $N=4096$	1.00	0.79	$1.26\times$	32%

WMMA’s implementation lift survives MPI: at $N=16K$, $P=4$, FP16 WMMA runs in 1.56 ms versus 4.90 ms for FP32 chunked ($3.14\times$, modestly below the $4.65\times$ single-GPU lift because some of the WMMA win is now spent on halo communication). Efficiency drops with FP16 because halo volume $4wHd$ is unchanged but per-rank compute shrunk; efficiency recovers as N grows (32%, 47%, and 64% at $N=4K$, 8K, and 16K), exactly the $W \sim N \sim P$ isoefficiency curve.

I Algorithm pseudocode

Shared kernel skeleton. All three kernels share the fused-tile, register-resident online-softmax structure in Algorithm 1; the *only* place they differ is in the iterator $\mathcal{I}(i)$ over K/V tile indices.

Iterator $\mathcal{I}(i)$ — the only difference among the three kernels:

- **Dense:** $\mathcal{I}(i) = \{0, 1, \dots, \lceil N/B_c \rceil - 1\}$ (every K/V tile)
- **Windowed** (half-width w , global tokens $[0, G)$):

$$\mathcal{I}(i) = \{j : [jB_c, (j+1)B_c) \cap ([i-w, i+w] \cup [0, G)) \neq \emptyset\}$$

- **Block-sparse** (CSR mask with `row_ptr/col_idx`, block size B):

$$\mathcal{I}(i) = \text{col_idx}[\text{row_ptr}[b] : \text{row_ptr}[b+1]] \quad \text{where } b = \lfloor i/B \rfloor$$

Algorithm 1 Fused-tile attention kernel (shared by all three variants).

Require: Query tile $Q_i \in \mathbb{R}^{B_r \times d}$ at row block i ; full $K, V \in \mathbb{R}^{N \times d}$; iterator $\mathcal{I}(i)$ over K/V tile indices

Ensure: Output rows $O_i \in \mathbb{R}^{B_r \times d}$

```

1:  $m_i \leftarrow -\infty, \ell_i \leftarrow 0, O_i \leftarrow 0$  ▷ per-row state in registers
2: for each tile index  $j \in \mathcal{I}(i)$  do
3:   Cooperatively load  $K_j, V_j$  into SMEM via float4 loads
4:    $s_j \leftarrow Q_i \cdot K_j^\top / \sqrt{d}$  ▷  $B_r \times B_c$  score block
5:    $m_{\text{new}} \leftarrow \max(m_i, \max_j s_j)$ 
6:    $\alpha \leftarrow \exp(m_i - m_{\text{new}}), \beta_j \leftarrow \exp(s_j - m_{\text{new}})$ 
7:    $\ell_i \leftarrow \alpha \ell_i + \sum_j \beta_j$ 
8:    $O_i \leftarrow \alpha O_i + \beta_j V_j$ 
9:    $m_i \leftarrow m_{\text{new}}$ 
10: end for
11:  $O_i \leftarrow O_i / \ell_i$  ▷ final normalization

```

Sequence-parallel MPI driver. Algorithm 2 shows the per-iteration structure of the seq-par windowed driver: pack halo edges, exchange via four `MPI_Sendrecv` calls, then launch the windowed kernel on the halo-extended local buffers.

Algorithm 2 Sequence-parallel windowed MPI driver (one iteration on rank r).

Require: Local Q_r and O_r owning rows $[rN/P, (r+1)N/P)$; local K_r, V_r buffers with w -row halo regions on each side

```

1: ▷ Halo exchange: 4 MPI_Sendrecv calls total
2: for each tensor  $T \in \{K, V\}$  do
3:   Pack right edge of  $T_r$  ( $w$  rows  $\times H$  heads  $\times d$ ) via cudaMemcpy2D into a contiguous send buffer
4:   MPI_Sendrecv: send right-edge to rank  $r+1$ ; receive from rank  $r-1$  into local left halo
5:   Pack left edge of  $T_r$  similarly
6:   MPI_Sendrecv: send left-edge to rank  $r-1$ ; receive from rank  $r+1$  into local right halo
7: end for
8: ▷ Local kernel launch
9: Run Algorithm 1 on  $Q_r$  with the halo-extended  $K_r, V_r$  using the windowed iterator
10: (pass local_offset = w so global query rows map to the local  $K$  frame)

```
